How to Write a Smart Contract in Solidity & Deploy to Sepolia Testnet

GU Zhan (Sam)
sam.gu@ntu.edu.sg
(+65) 9011 2860



A Brief History of Blockchain & Decentralization

The Double Spend Problem

Definition: The risk that a digital currency can be spent more than once

Traditional Approach: Typically prevented by central authorities (e.g., banks)

Challenge for Decentralization:

- Removing the central authority meant finding a new solution to verify and secure transactions
- Critical to ensuring trust in peer-to-peer digital currency systems

1991 – The Foundation

Milestone: First description of a cryptographically secured chain of blocks

Key Contributors: Stuart Haber and W. Scott Stornetta

Publication: "How to Time-Stamp a Digital Document"

Significance: Introduced the idea of linking records in a tamper-resistant way

1998 – Nick Szabo & Bit Gold

Milestone: Nick Szabo conceptualizes “bit gold”

Goal: A decentralized digital currency

Key Insight:

- Proposed a mechanism to prevent double spending without a central authority
- Influenced Bitcoin’s eventual design

2000 – Stefan Konst’s Contributions

Publication: Stefan Konst’s theory of cryptographic secured chains

Ideas:

- Ensuring data integrity over time using cryptography
- Offered proposals on how to implement secure, chained data structures
- Impact: Further shaped the concept of immutable ledgers

2008 – Satoshi Nakamoto’s White Paper

Milestone: "Bitcoin: A Peer-to-Peer Electronic Cash System"

Author: Pseudonymous developer(s) “Satoshi Nakamoto”

Key Contributions:

- Detailed method for decentralized currency transactions
- Introduced Proof of Work for network security

Relevance: Blueprint for all subsequent blockchain projects

2009 – The First Blockchain Implementation

Launch: Bitcoin public ledger by Nakamoto

Breakthrough:

- Solved Double Spend Problem in a decentralized manner
- Network of peers validate and secure transaction data

Result: Start of modern cryptocurrency and blockchain ecosystems

2014 – Ethereum and Smart Contracts

Milestone: Ethereum blockchain introduced

Innovation: Smart Contracts – self-executing programs on the blockchain

Implications:

- Beyond simple value transfers
- Powered decentralized applications (dApps)

2021 – Web3 Emerges

Concept: A new iteration of the Web emphasizing decentralization, blockchain, and token-based economics

Vision:

- User-owned networks, greater transparency
- Redefined data ownership and governance

Future Outlook: Continues evolving, shaping online info & asset exchange

Workshop

How to Write a Smart Contract in Solidity & Deploy to Sepolia Testnet

Goals to achieve

1. Writing a smart contract (dApp), in Solidity programming language, using Remix development tool on the cloud

2. Deploying the dApp on Sepolia testnet (Ethereum test network)

Why Solidity & Smart Contracts?

- Smart Contracts automate agreements and transactions on the blockchain.
- Solidity is the primary language for Ethereum, designed for secure dApps.

Setting Up Your Development Environment

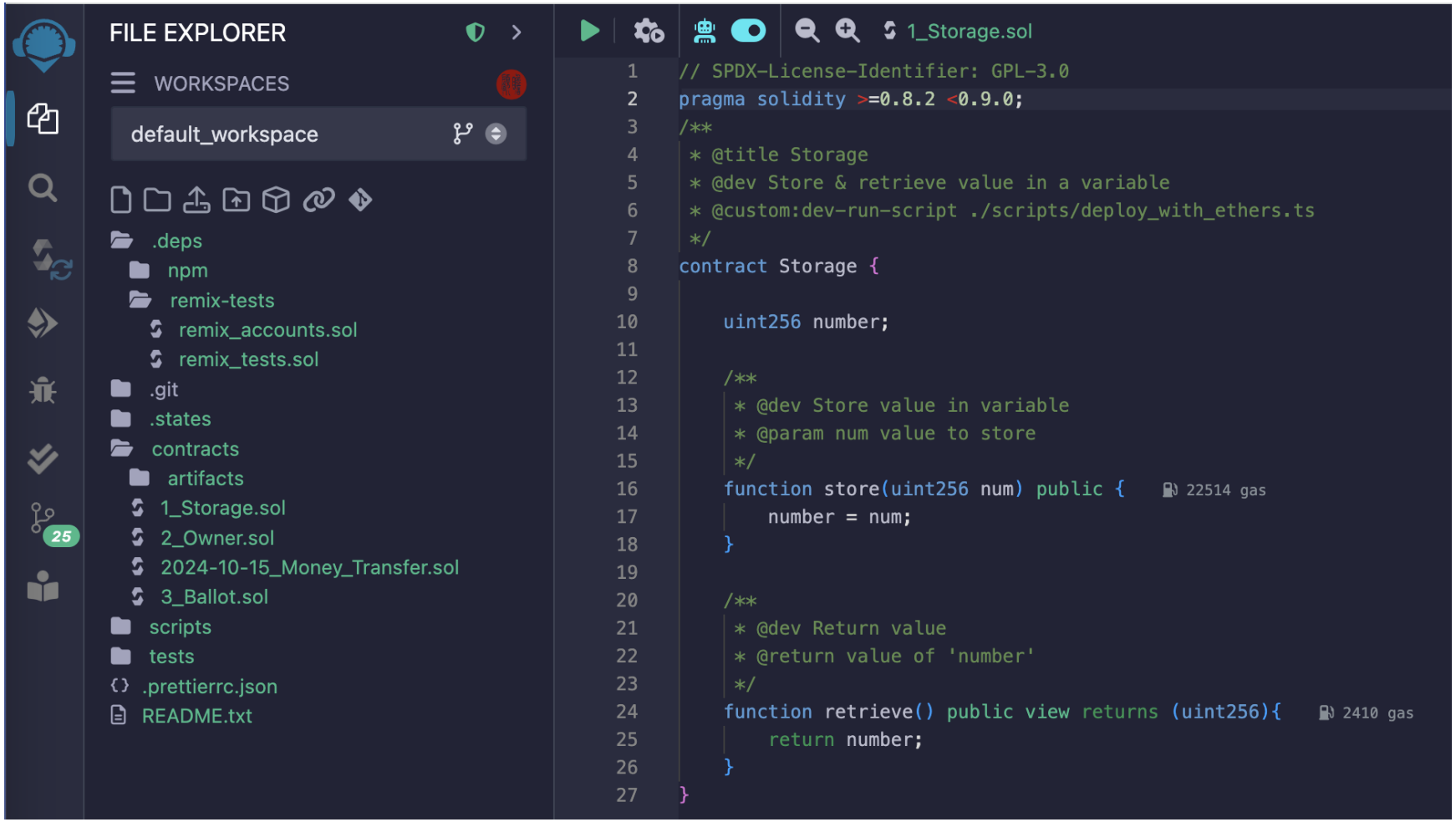
- Remix IDE: Web-based, great for beginners, no installation required.
- VS Code + Solidity: Local dev environment for bigger projects.
- Truffle Suite & Ganache: Advanced setup, local blockchain simulation.

Key Solidity Concepts

- Contracts: Like classes, contain code and data.
- State Variables: Permanently stored on the blockchain.
- Functions: Execute specific logic.
- Modifiers: Enhance or restrict functions.
- Events: For logging, off-chain reading.

Writing Your First Contract

https://remix.ethereum.org/



Deploying Locally in Remix

- Open remix.ethereum.org, create/open .sol file
- Select compiler version ≥ 0.8.0, click Compile
- Deploy with Remix VM, see Deployed Contracts

Introduction to Sepolia Testnet

- A public Ethereum test network for safe experiments
- Free test Ether from faucets, similar to mainnet environment

Configuring Metamask for Sepolia

- Install Metamask extension
- Switch to Sepolia test net in Metamask

Getting Test Ether

- Visit a known Sepolia faucet to request free ETH
- Confirm the balance in your Metamask wallet

Deploying on Sepolia from Remix

- Mtehod 1: Select ‘Remix VM' in Remix
- Method 2: Select 'Injected Provider - Metamask’ or ‘Sepolia Testnet - Metamask’ in Remix
- Deploy and approve transaction in Metamask
- Wait for transaction mined confirmation

DEPLOY & RUN TRANSACTIONS

ENVIRONMENT Reset State

Remix VM (Cancun)

Injected Provider - MetaMask
Sepolia Testnet - MetaMask

Remix VM (Cancun)
Remix VM - Mainnet fork

WalletConnect
Custom - External Http Provider
Dev - Hardhat Provider
Dev - Foundry Provider

Customize this list...

Deploy

Publish to IPFS

At Address | Load contract from Addre

Transactions recorded 3

1_Storage.sol

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.8.2 <0.9.0;
3  /**
4   * @title Storage
5   * @dev Store & retrieve value in a variable
6   * @custom:dev-run-script ./scripts/deploy_with_ethers.ts
7   */
8  contract Storage {
9
10     uint256 number;
11
12     /**
13      * @dev Store value in variable
14      * @param num value to store
15      */
16     function store(uint256 num) public {
```

0 Listen on all transactions Filter with transaction hash or address

call to Storage.retrieve

CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Storage.retrieve()
data: 0x2e6...4cec1 Debug

transact to Storage.store pending ...

[vm] from: 0x5B3...eddC4 to: Storage.store(uint256) 0x540...c7569 value: 0 wei
data: 0x605...875c1 logs: 0 hash: 0x564...4dba3 Debug

call to Storage.retrieve

CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Storage.retrieve()
data: 0x2e6...4cec1 Debug

>

Deploying on Sepolia from Remix

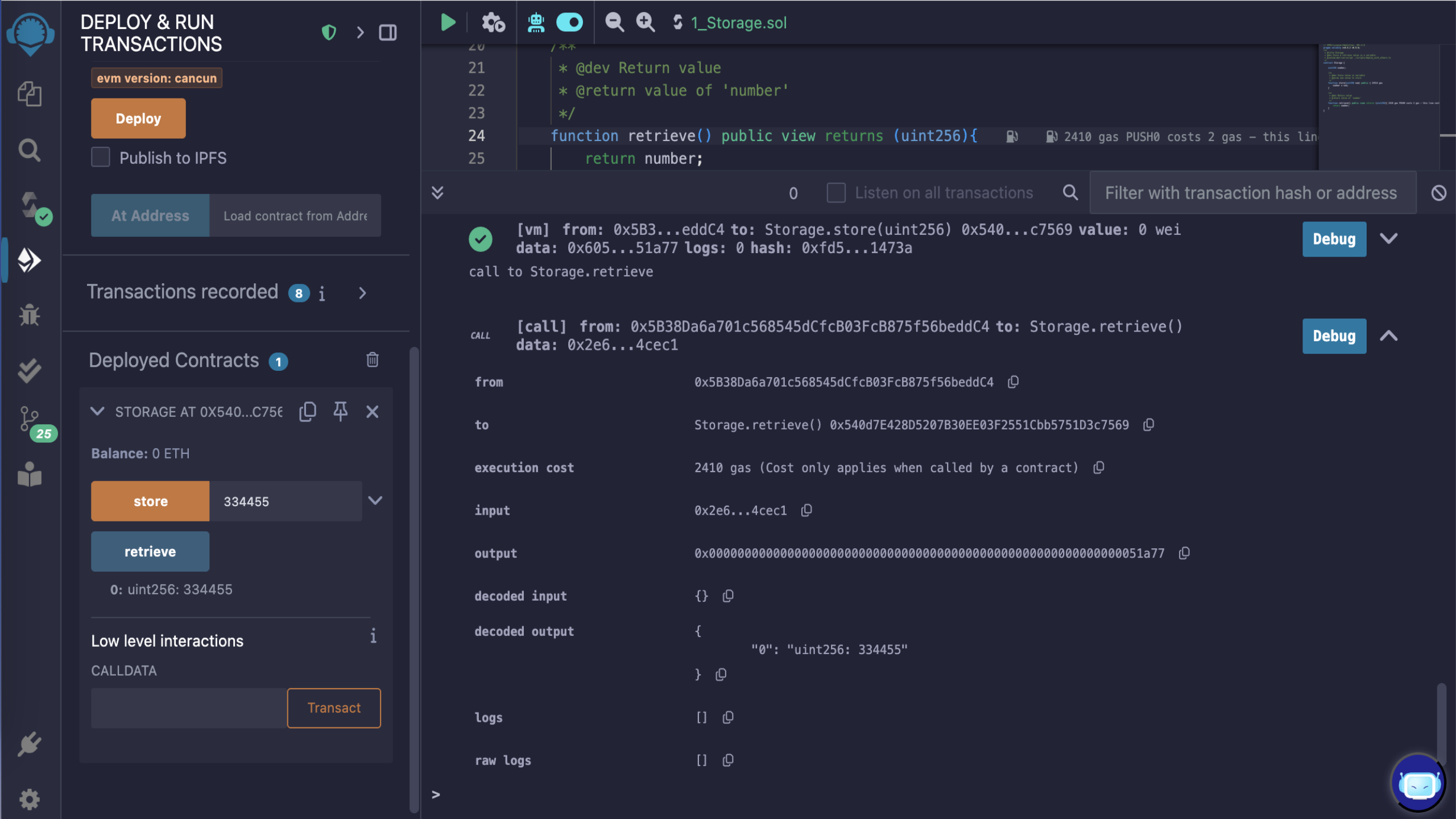
- **Remix VM**: Local, simulated blockchain; no real network or wallet needed; instant, cost-free transactions.
- **Injected Provider - Metamask**: Uses Metamask’s current network (could be Mainnet, Goerli, Sepolia, etc.); transactions require funds on that network.
- **Sepolia Testnet - Metamask**: A specific case of the injected provider set to the **Sepolia** test network; you deploy with test ETH, ideal for safe real-world testing.

Verifying Deployment

- Check Remix console for success message
- Use transaction hash on sepolia.etherscan.io to confirm

Interacting with Your Deployed Contract

- Expand contract in Remix to see functions
- store() changes state and uses gas, retrieve() is free
- Metamask will prompt confirmation for paid operations


DEPLOY & RUN TRANSACTIONS
evm version: cancun
Deploy
Publish to IPFS
At Address
Load contract from Addre
Transactions recorded 8
Deployed Contracts 1
STORAGE AT 0X540...C756
Balance: 0 ETH
store
334455
retrieve
0: uint256: 334455
Low level interactions
CALLDATA
Transact
1_Storage.sol
21 * @dev Return value
22 * @return value of 'number'
23 */
24 function retrieve() public view returns (uint256){
25 return number;
2410 gas PUSH0 costs 2 gas - this lin
0
Listen on all transactions
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: Storage.store(uint256) 0x540...c7569 value: 0 wei
data: 0x605...51a77 logs: 0 hash: 0xfd5...1473a
Debug
call to Storage.retrieve
CALL
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Storage.retrieve()
data: 0x2e6...4cec1
Debug
from 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to Storage.retrieve() 0x540d7E428D5207B30EE03F2551Cbb5751D3c7569
execution cost 2410 gas (Cost only applies when called by a contract)
input 0x2e6...4cec1
output 0x0000000000000000000000000000000000000000000000000000000000051a77
decoded input {}
decoded output {
"0": "uint256: 334455"
}
logs []
raw logs []
>

Common Troubleshooting Tips

- If low on test ETH, return to faucet
- Verify Sepolia is selected in Metamask, not mainnet
- Match pragma solidity version with Remix compiler

The End, is a New Start.

References

1) How to Write a Smart Contract in Solidity: A Beginner’s Guide
https://medium.com/kalpstudio/how-to-write-a-smart-contract-in-solidity-a-beginners-guide-fba95dc886b4

2) Deploying Your Smart Contract to Testnet (Sepolia) Using Metamask
https://medium.com/kalpstudio/deploying-your-smart-contract-to-testnet-sepolia-using-metamask-8f0916721f8b